Machine Translated by Google

Decision Tree Classification Example (1)

ÿ Data for predicting bank credit card customer churn: “Credit Card customers

ÿ Provided by Kaggle, www.Kaggle.com/datasets/sakshigoyal7/credit-card-customers

ÿ Attributes included in bank_churners.csv

열 이름	설명
CLIENTNUM	고객번호
Attrition_Flag	이탈고객여부. 0=유지, 1=이탈
Customer_Age	고객 나이
Gender	고객 성별. 0=남성, 1=여성
Dependent_count	부양가족 수
Income_Category	연소득 수준. 0=자료없음, 1=$40K 미만, 2=$40K – 60K, 3=$60K – $80K, 4=$80K-$120K, 5=$120K 초과
Card_Category	카드 등급. 0=Blue, 1=Silver, 2=Gold, 3=Platinum
Months_on_book	은행과의 총 거래기간
Total_Relationship_Count	은행 상품 보유 갯수
Contacts_Count_12_mon	최근 12개월간 은행과의 접촉 수
Credit_limit	신용 한도
Total_Revolving_Bal	리볼빙 잔액
Avg_Open_To_Buy	신용 잔여액
Total_Trans_Amt	평균 거래금액
Total_Amt_Chng_Q4_Q1	거래금액 변화량 (Q4 대비 Q1)

Machine Translated by Google

Decision Tree Classification Examples (2)

ÿ Determining whether a customer has churned based on customer feat

ÿ Factors to consider when analyzing customer churn

Selection of Traits (13) ÿ

Churn Status: Attrition_Flag ÿ Analysis Method:

Through Decision Trees

Classification analysis

- Customer_Age
- Gender
- Dependent_count
- Income_Category
- Card_Category
- Months_on_book
- Total_Relationship_Count
- Contacts_Count_12_mon
- Credit_limit
- Total_Revolving_Bal
- Avg_Open_To_Buy
- Total_Trans_Amt
- Total_Amt_Chng_Q4_Q1

Machine Translated by Google

Decision Tree Classification Examples (3)

ÿ Read the data file and construct a data frame

ÿ Data file: bank_churners.csv ÿ Data frame containing all features: df ÿ Construct the following data frame from df ÿ 13 customer features: input_feature ÿ Customer churn status: target

```
import pandas as pd
df = pd.read_csv('bank_churners.csv')

input_feature = df[['Customer_Age', 'Gender',
  'Dependent_count', 'Income_Category', 'Card_Category',
  'Months_on_book', 'Total_Relationship_Count',
  'Contacts_Count_12_mon', 'Credit_Limit',
  'Total_Revolving_Bal', 'Avg_Open_To_Buy',
  'Total_Trans_Amt', 'Total_Amt_Chng_Q4_Q1']]

target = df['Attrition_Flag']
```

ÿ Separate the entire training data

ÿ Use the train_test_split function

ÿ Split training and evaluation data in an 8:2 ratio ÿ Check the number of split data points

ÿ Total 10,127 cases ÿ Training 8,101 cases ÿ Evaluation 2,026 cases

Training(x) 8,101 cases
Evaluation(x) 2,026 cases Training(y) 8,101 cases Evaluation(y) 2,026 cases

```
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test =
    train_test_split(input_feature, target,
    test_size=0.2, random_state=123)

print("Training (x) {} cases".format(len(x_train)))
print("Evaluation (x) {} cases".format(len(x_test)))
print("Training (y) {} cases".format(len(y_train)))
print("Evaluation (y) {} cases".format(len(y_test)))
```

Machine Translated by Google

Decision Tree Classification Examples (4)

ÿ Create a decision tree model

ÿ Import required features from sklearn, model_name dt_class, max_depth(max_depth) = 3

ÿ Construct the model (dt_class) using training data

ÿ Construct a decision tree using fit(ÿ)

ÿ Generate predicted values using test data (x_test)

ÿ Predicted value : y_pred

```
from sklearn.tree import DecisionTreeClassifier

dt_class = DecisionTreeClassifier(max_depth=3,
                                  random_state=123)

dt_class.fit(x_train,y_train)

y_pred = dt_class.predict(x_test)
```

ÿ Performance Evaluation

ÿ Creation of a confusion matrix

ÿ Check accuracy

ÿ Check Classification Evaluation Table

```
from sklearn import metrics

confusion_matrix = metrics.confusion_matrix(y_test, y_pred)
accuracy = metrics.accuracy_score(y_test, y_pred)
print(confusion_matrix) # Create misclassification table
print(accuracy)                    # Check accuracy
print(metrics.classification_report(y_test, y_pred))
                                   # Check Classification Evaluation Table
```

```
[[1616 99]
 [ 116 195]]
0.8938795656465943
              precision recall f1-score support

           0       0.93      0.94      0.94      1715
           1       0.66      0.63      0.64       311

    accuracy                           0.89      2026
   macro avg       0.80      0.78      0.79      2026
weighted avg       0.89      0.89      0.89      2026
```

Machine Translated by Google

Decision Tree Classification Examples (5)

ÿ Visualization processing for the decision tree result (dt_class) obtained above

ÿ Utilizing the graphics tool Graphviz ÿ The

Graphviz package is required for decision tree visualization ÿ **From https://**

graphviz.gitlab.io/download/ Download/Install -graphviz-2.49.3 (64-bit)

EXE installer

ÿ Connecting the Graphviz package to Python

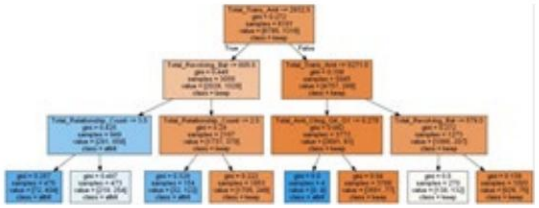
```
import
os os.environ["PATH"] += os.pathsep + 'C:/Program Files/
GraphViz/bin' !pip install graphviz
```

ÿ Setting classification values (Attrition_Flag values), organizing input feature column names

```
target_names = ['keep', 'attrit']
class_names=input_feature.columns.to_list()
```

ÿ Visualize the decision tree result (dt_clf) obtained earlier

```
from sklearn.tree import export_graphviz
export_graphviz(dt_class, out_file="tree.dot",
        class_names=target_names,
        feature_names=class_names, impurity = True, filled =
True) import
graphviz with
  open("tree.dot") as
f: dot_graph = f.read() graphviz.Source(dot_graph)
```



화면상에서 처리

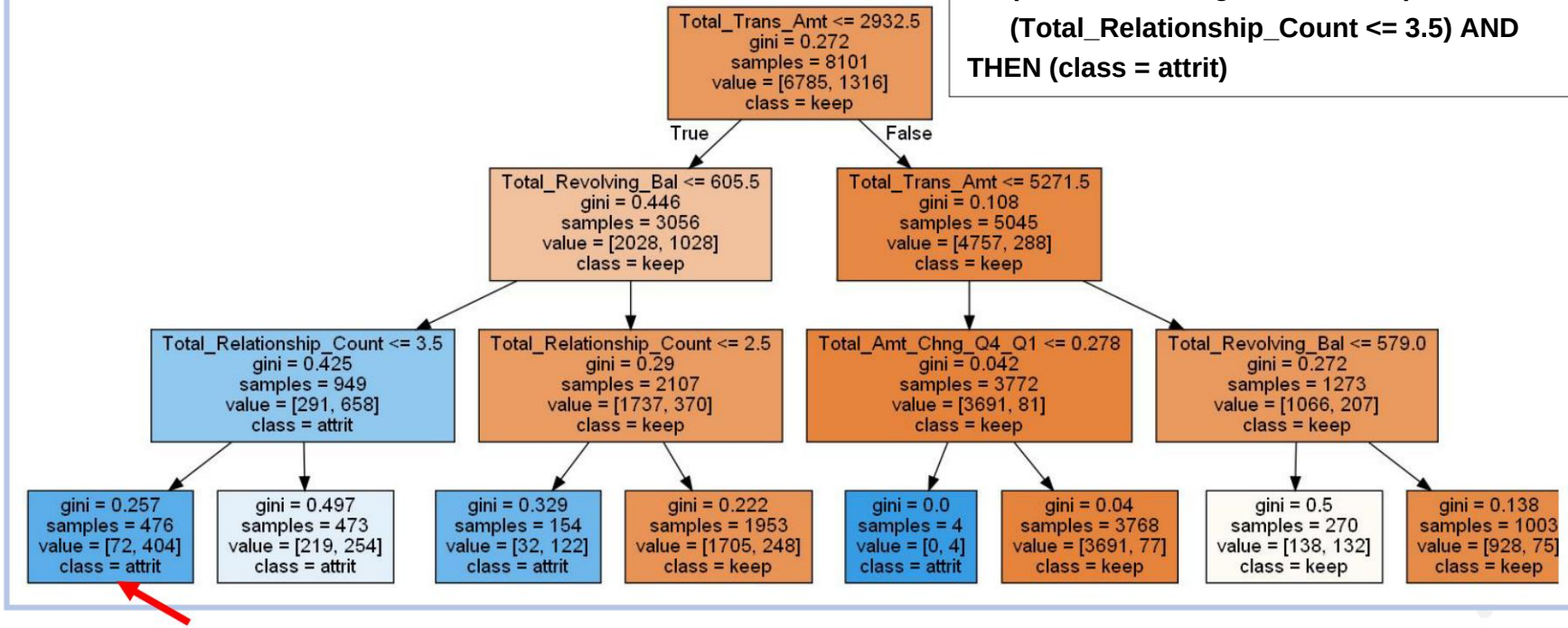
Machine Translated by Google

Decision Tree Classification Examples (6)

ÿ Save the visualized decision tree fi

```
graph = graphviz.Source(dot_graph)
graph.render(filename='test_img', directory='./', format='jpg')
```

ÿ Save to “test_img.jpg” file

< Rule Example: 1st terminal node from the left >

IF (Tota;_Trans_Amt <= 2932.5) AND
(Total_Revolving_Bal <= 605.5) AND
(Total_Relationship_Count <= 3.5) AND
THEN (class = attrit)